

Modul Belajar: Safe Reinforcement Learning dari Policy Gradient sampai PPO-Lagrangian

Catatan studi — MountainCar-v0

June 5, 2026

Contents

1	Gambaran Besar	1
2	Fondasi: Markov Decision Process	1
2.1	Return dan Discount	1
3	Policy dan Policy Gradient	2
3.1	Teorema Policy Gradient	2
4	Value Function dan Advantage	2
4.1	Generalized Advantage Estimation (GAE)	2
5	PPO: Proximal Policy Optimization	3
5.1	Probability Ratio	3
5.2	Clipped Surrogate Objective	3
5.3	Loss Value Function	3
5.4	Langkah Update: Di Mana $1r$ Muncul	3
5.5	Satu Epoch PPO	4
6	Safe PPO: PPO-Lagrangian	4
6.1	Lagrangian	4
6.2	Update Dual untuk λ	4
7	Studi Kasus: Kenapa λ Tidak Tumbuh	4
7.1	Gejala	4
7.2	Diagnosa: Self-Suppression	5
7.3	Penyebab Collapse Seed 1	5
7.4	Perbaikan Minimal	5
8	Ringkasan Hyperparameter	5
9	Ringkasan Simbol	6

1 Gambaran Besar

Tujuan project: melatih sebuah *agen* menyelesaikan game **MountainCar-v0**, namun **sambil mematuhi sebuah batasan keselamatan** (jangan ngebut melebihi ambang kecepatan tertentu). Ini adalah masalah *Safe Reinforcement Learning*: memaksimalkan hadiah sekaligus menjaga sebuah biaya pelanggaran tetap di bawah batas.

Dua tujuan yang saling bertarung:

- *Reward* — cepat mencapai puncak bukit.
- *Cost* — jangan melampaui batas kecepatan.

Karena mobil **butuh** kecepatan untuk menanjak, kedua tujuan ini secara intrinsik tegang. Di sinilah letak kesulitan (dan keindahan) masalahnya.

2 Fondasi: Markov Decision Process

RL dimodelkan sebagai **Markov Decision Process** (MDP), tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$:

- $\mathcal{S}$ — himpunan **state** (mis. posisi & kecepatan mobil).
- $\mathcal{A}$ — himpunan **action** (mis. dorong kiri / diam / kanan).
- $P(s' | s, a)$ — probabilitas transisi ke state s' setelah aksi a .
- $r(s, a)$ — **reward** yang diterima.
- $\gamma \in [0, 1)$ — **discount factor**.

Loop interaksinya berulang:

lihat $s_t \rightarrow$ pilih $a_t \rightarrow$ terima $r_t, s_{t+1} \rightarrow \dots$

2.1 Return dan Discount

Yang dimaksimalkan bukan reward sesaat, melainkan **return** (total reward terdiskon):

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}.$$

Faktor γ^k membuat reward jauh di masa depan bernilai lebih kecil, sekaligus menjaga jumlahnya tetap berhingga.

Kadang aksi yang reward-nya jelek *sekarang* membuka jalan ke reward besar *nanti* — persis mobil yang harus mundur dulu sebelum bisa menanjak. Itu sebabnya kita memaksimalkan *return*, bukan reward sesaat.

3 Policy dan Policy Gradient

Policy adalah strategi agen: sebuah distribusi peluang atas aksi,

$$\pi_{\theta}(a | s),$$

diparametrikkan oleh θ (bobot jaringan neural). Belajar = menyetel θ .

Objektif yang dimaksimalkan:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t \gamma^t r_t \right],$$

dengan τ sebuah *trajectory* (satu jalannya game).

3.1 Teorema Policy Gradient

$$\nabla_{\theta} J(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A_t]$$

- $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ — arah yang **menaikkan peluang** aksi a_t .
- A_t — **advantage**, bobot yang menentukan seberapa kuat aksi itu didorong.

Aturannya satu kalimat: *kalau suatu aksi ternyata lebih bagus dari perkiraan ($A_t > 0$), perbesar peluangnya; kalau lebih jelek ($A_t < 0$), perkecil*. Agen tidak butuh "jawaban benar" — ia membandingkan kenyataan dengan ekspektasinya sendiri.

4 Value Function dan Advantage

Value function $V_\phi(s)$ (si *critic*, jaringan neural kedua) menebak return mulai dari state s :

$$V(s) = \mathbb{E}[G_t \mid s_t = s].$$

Advantage mengukur seberapa lebih bagus suatu aksi dibanding rata-rata di state itu:

$$A_t = Q(s_t, a_t) - V(s_t).$$

Dalam praktik dipakai **TD error** (estimasi 1-langkah):

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t),$$

yaitu (reward nyata + nilai state berikutnya) – (tebakan nilai sekarang) — sebuah "kejutan".

4.1 Generalized Advantage Estimation (GAE)

PPO menghaluskan advantage sebagai rata-rata berbobot dari TD error:

$$A_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma \lambda_{\text{gae}})^l \delta_{t+l},$$

dengan $\lambda_{\text{gae}} \in [0, 1]$ menyeimbangkan *bias* vs *variance*. (Catatan: λ_{gae} ini berbeda dari λ Lagrangian di Bagian 6.)

5 PPO: Proximal Policy Optimization

Policy gradient mentah rapuh: **satu update terlalu besar bisa merusak policy secara permanen**. PPO mengatasinya dengan prinsip *tetap dekat* dengan policy lama.

5.1 Probability Ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

- $r_t = 1$ — policy tidak berubah.
- $r_t = 1.2$ — policy baru 20% lebih sering memilih aksi tersebut.

5.2 Clipped Surrogate Objective

$$L^{\text{CLIP}}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) A_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) A_t \right) \right]$$

dengan ϵ (mis. 0.2) lebar batas. Mekanisme **min+clip**:

- **Jika** $A_t > 0$ (aksi bagus): suku ter-clip memberi *plafon* $1.2 A_t$. Begitu r_t melewati 1.2, objektif berhenti naik $\Rightarrow$ gradien nol $\Rightarrow$ tak ada insentif melompat lebih jauh.
- **Jika** $A_t < 0$ (aksi jelek): **min** memilih suku yang lebih pesimis, koreksi tetap diberikan namun terkendali.

Seperti memperbaiki resep dari feedback pelanggan dengan aturan "*maksimal ubah satu sendok bumbu per percobaan*": pelan, tapi stabil dan tak pernah merusak resep dalam semalam. Itulah seluruh rahasia kestabilan PPO.

5.3 Loss Value Function

Critic dilatih dengan kuadrat selisih terhadap return nyata R_t :

$$L^V(\phi) = \mathbb{E}[(V_\phi(s_t) - R_t)^2].$$

5.4 Langkah Update: Di Mana lr Muncul

Perhatikan: lr (learning rate) **tidak muncul** di rumus objektif mana pun (L^{CLIP} , L^V , teorema policy gradient). Itu wajar. Persamaan-persamaan tersebut hanya mendefinisikan *apa* yang dioptimalkan dan menghasilkan *arah* (gradien). lr baru masuk di langkah terpisah: *bagaimana* mengambil langkahnya.

$$\theta \leftarrow \theta + \alpha \nabla_\theta L^{\text{CLIP}}(\theta), \quad \phi \leftarrow \phi - \alpha_V \nabla_\phi L^V(\phi).$$

- $\nabla_\theta L$ — menentukan **arah** (ke mana naik).
- α ($=\text{lr}$) — menentukan **panjang langkah**.

Gradien itu **kompas** (menunjuk arah naik); lr itu **panjang langkahmu**. Objektif cuma menggambar petanya; lr adalah keputusan seberapa jauh melangkah tiap kali — terpisah dari peta. lr besar = cepat tapi mudah overshoot; lr kecil = stabil tapi lambat.

Bandingkan dengan update λ Lagrangian, yang aturannya ditulis *eksplisit*, sehingga laju belajarnya ($\eta = \text{lam_lr}$) **memang terlihat** di persamaan:

$$\log \lambda \leftarrow \log \lambda + \eta (J_C - d).$$

Untuk θ dan ϕ , langkah update biasanya disembunyikan di balik optimizer (Adam/SGD) — itu sebabnya lr -nya tak tampak di rumus objektif. Catatan: Adam juga memodifikasi langkah efektif per-parameter, jadi lr yang diset bukan persis panjang langkah akhir, tapi tetap berperan sebagai pengali besar-kecilnya update.

5.5 Satu Epoch PPO

1. Jalankan policy sekarang, kumpulkan N langkah pengalaman (mis. `steps_per_epoch=4000`).
2. Hitung advantage A_t (via GAE).
3. Naikkan L^{CLIP} terhadap θ (dengan rem clipping).
4. Turunkan L^V terhadap ϕ .
5. Ulangi sebanyak `epochs`.

6 Safe PPO: PPO-Lagrangian

Sekarang ada **dua** sinyal: reward menghasilkan A_t^R , dan cost (pelanggaran kecepatan) menghasilkan A_t^C . Masalahnya menjadi optimisasi berkendala:

$$\max_{\theta} J_R(\theta) \quad \text{s.t.} \quad J_C(\theta) \leq d,$$

dengan J_C ekspektasi cost dan d batasnya (`cost_limit=15`).

6.1 Lagrangian

Bentuk Lagrangian-nya memperkenalkan pengali $\lambda \geq 0$:

$$\mathcal{L}(\theta, \lambda) = J_R(\theta) - \lambda(J_C(\theta) - d).$$

Untuk policy, advantage gabungan yang dimasukkan ke rumus PPO menjadi:

$$A_t = A_t^R - \lambda A_t^C$$

λ adalah "**kenop tarif denda**" untuk ngebut. Jika λ kecil, suku λA_t^C nyaris nol $\Rightarrow$ agen mengabaikan cost $\Rightarrow$ ngebut terus. Jika λ besar, agen takut melanggar.

6.2 Update Dual untuk λ

λ dinaikkan lewat **gradient ascent** pada masalah dual:

$$\lambda \leftarrow [\lambda + \eta(J_C - d)]_+$$

dengan $\eta = \text{lam_lr}$ dan $[\cdot]_+$ proyeksi ke $\lambda \geq 0$. Selama melanggar ($J_C > d$), λ naik $\Rightarrow$ denda makin mahal, sampai J_C mendarat di d .

7 Studi Kasus: Kenapa λ Tidak Tumbuh

7.1 Gejala

- avg cost: 26–37 (target ≤ 15) — constraint dilanggar.
- λ final: 0.027–0.040 — sangat kecil, bahkan turun dari init ≈ 0.05 .
- Seed 1 return collapse ke 11 (seed lain ~ 44 –48).

7.2 Diagnosa: Self-Suppression

Implementasi menggunakan parametrisasi $\log \lambda$ dengan loss:

```
1 loss_lambda = -torch.exp(log_lambda) * (actual_batch_cost - cost_limit)
```

Gradiennya terhadap $\log \lambda$:

$$\frac{\partial \text{loss}}{\partial \log \lambda} = -\exp(\log \lambda)(J_C - d) = -\lambda(J_C - d).$$

Faktor $\lambda \approx 0.05$ ikut **mengalikan setiap update**. Langkah efektif di ruang log:

$$\Delta \log \lambda \approx \eta \lambda (J_C - d) \approx 0.005 \times 0.05 \times 15 \approx 0.004 \text{ per epoch.}$$

Self-suppression: justru saat λ kecil, ia menahan pertumbuhannya sendiri. Di epoch awal (mobil belum ngebut, $J_C < d$) λ malah didorong *turun*, lalu terjebak di nilai sangat kecil — persis pola λ final yang lebih kecil dari nilai inisial.

7.3 Penyebab Collapse Seed 1

MountainCar butuh $|v|$ hingga ~ 0.07 untuk mencapai goal, sedangkan cost menghukum $|v| > 0.04$. Constraint dan task **saling bertentangan**. Saat λ secara lokal menang, agen mengorbankan goal demi menurunkan kecepatan $\Rightarrow$ return jatuh. Ini tension Pareto cost-vs-reward, diperparah osilasi Lagrangian.

7.4 Perbaikan Minimal

(a) Hilangkan faktor $\exp$, normalisasi violation, tambah clamp:

```
1 with torch.no_grad():
2     violation = (actual_batch_cost - cost_limit) / cost_limit # ternormalisasi
3     log_lambda += lam_lr * violation # tanpa faktor exp -> no self-
        suppression
4     log_lambda.clamp_(min=-2.0, max=3.0) # floor & ceiling
5 lam = torch.exp(log_lambda)
```

(b) Hyperparameter:

$\text{lam_lr} : 0.005 \rightarrow 0.03$, $\text{log_lambda_init} : -3.0 \rightarrow 0.0$ ($\lambda \approx 1.0$).

Alasan: mulai dari λ yang sebanding skala reward agar constraint langsung "menggigit"; lam_lr cukup besar agar responsif dalam 150 epoch tanpa overshoot liar; normalisasi violation memisahkan lam_lr dari skala cost mentah.

Peringatan jujur: karena ambang 0.04 melawan kebutuhan velocity, target avg cost ≤ 15 mungkin menuntut pengorbanan sebagian return. Kalau setelah fix return ikut turun di semua seed, itu sinyal cost_limit /ambang yang perlu dilonggarkan — bukan lagi soal tuning λ .

8 Ringkasan Hyperparameter

Tiga kategori besaran — jangan tertukar: *hyperparameter* kamu set sendiri, *parameter* disetel training, *besaran* dihitung dari data.

Kategori	Contoh	Asalnya
Hyperparameter (PPO)	$\gamma, \text{lr}, \epsilon, \text{lam_gae}, \text{ent_coef}$	kamu ketik di config
Hyperparameter (safety)	$\text{cost_limit}, \text{lam_lr}, \text{log_lambda_init}$	kamu ketik di config
Parameter dipelajari	θ (policy), ϕ (critic), λ (Lagrangian)	disetel saat training
Besaran dihitung	$\tau, \delta_t, A_t, r_t, G_t$	dihitung dari data

Aturan cepat membedakan: kalau **dihitung dari data** ($A_t, \delta_t, r_t, \tau, G_t$) $\Rightarrow$ bukan hyperparameter. Kalau **disetel jaringan saat training** (θ, ϕ) $\Rightarrow$ parameter yang dipelajari. Kalau **kamu ketik angkanya di config sebelum run** $\Rightarrow$ baru itu hyperparameter.

9 Ringkasan Simbol

Simbol	Makna
$\pi_{\theta}(a \mid s)$	policy: peluang aksi a di state s
θ, ϕ	bobot jaringan policy (actor) & value (critic)
G_t, R_t	return (total reward terdiskon)
γ	discount factor
$V(s), A_t$	value function & advantage
δ_t	TD error
$r_t(\theta)$	probability ratio (policy baru / lama)
ϵ	lebar clipping PPO
α, α_V	learning rate policy & critic (<code>lr</code>)
λ	pengali Lagrangian (denda cost)
λ_{gae}	parameter GAE (bias-variance)
J_C, d	ekspektasi cost & batasnya (<code>cost_limit</code>)
η	laju belajar λ (<code>lam_lr</code>)